Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Department of Computer Science, University of Oviedo, Gijón, Spain

Abstract

Dispatching rules remain the method of choice for scheduling under tight time budgets, but hand-crafted rules leave substantial performance unexploited, and the automated design of rules has scarcely been explored for scheduling problems with non-deterministic processing times. We present a genetic programming (GP) hyper-heuristic for the job shop scheduling problem with interval durations (IJSP), in which operation durations are known only to lie within an interval and schedules are evaluated by interval arithmetic. The GP evolves priority expressions over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, so that evolved rules can react to uncertainty itself. On the 70 interval versions of Taillard's benchmark, over 30 independent evolutions the resulting rules average a relative error of 18.7% ($\pm$0.8) with a single deterministic constructive pass, and the best evolved rule attains 17.3%—improving on the strongest deterministic constructive baseline (Giffler–Thompson with MWKR tie-breaking, 29.5%) on every single instance—and generalizes zero-shot across all seven instance size classes despite being evolved on only four training instances of a single size class (20$\times$15). Embedded in a GRASP-style randomized scheme, the evolved rule converts sampling budget into

solution quality (13.7% at best-of-1024). Evolved rules are compact, human-readable expressions, and an automatic configuration study with irace shows the approach is robust to its own hyperparameters, with a preference for *smaller* expression trees at no performance cost. We position the method against fixed rules, active schedule generators, and published metaheuristics, characterizing the computational class in which interpretable evolved rules operate.

Keywords: interval job shop scheduling, genetic programming, hyper-heuristics, dispatching rules, scheduling under uncertainty, robustness

---

1. Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, schedules are built and evaluated with interval arithmetic, and solutions are compared by their worst-case (upper-bound) makespan. Population metaheuristics for the IJSP—artificial bee colony variants and tabu search among them—achieve strong results at the cost of minutes of per-instance search [2, 3, 4].

At the opposite end of the computational spectrum live *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. Their appeal is threefold—speed, interpretability, and trivial deployment—but hand-crafted rules (SPT, MOR, MWKR and relatives) leave a large quality gap: on the benchmark used in this paper the best fixed rule averages 45.5% relative error, versus 4–10% for metaheuristics.

For *deterministic* scheduling, a rich literature has shown that this gap can be narrowed substantially by evolving rules automatically with genetic programming (GP) [10, 11]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. It is not obvious a priori what an evolved rule should do with uncertainty: ignore it and rank by worst case, hedge against wide intervals, or exploit width information in a structured way. Making this possible requires exposing uncertainty to the rule as first-class attributes.

This paper makes the following contributions:

1. A GP hyper-heuristic for the IJSP whose terminal set is *interval-aware*: alongside worst-case processing time, earliest start and work remaining, rules can read the *widths* of these quantities, enabling uncertainty-sensitive prioritization (Section 4).
2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) beat every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes—evolved on 20×15, best class 50×15—and (iii) extend naturally to a randomized multi-start regime with monotone quality gains up to best-of-1024 (Section 6).
3. An analysis of *why* the rules work: an anatomy of the evolved expressions (Section 7.1), an ablation isolating the contribution of the interval-width terminals (Section 7.2), and an executional-robustness study showing that the evolved rule yields schedules whose realized makespan deviates less from its prediction than the baselines', increasingly so as uncertainty grows (Section 7.3).
4. An automatic configuration study (irace, 200 experiments) of the evolu-

tionary hyperparameters, confirmed over 30 independent evolutions per configuration: tuning brings no significant quality gain—the method is robust to its own knobs—but does yield systematically smaller, more readable trees at equal performance (Section 7.5).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 details the benchmark, the training protocol and the irace configuration study. Section 6 reports the empirical results, and Section 7 analyses them: rule anatomy, the interval-awareness ablation, executional robustness, transfer to crisp and fuzzy uncertainty models, hyperparameter sensitivity and limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [19]. Prior IJSP work is dominated by population-based metaheuristics: a genetic algorithm with a systematic study of interval rankings [1], artificial bee colony variants [2, 3], and a population-based tabu search that currently defines the state of the art [4]. All operate in the minutes-per-instance regime. Fast constructive methods for the IJSP have received far less attention; dispatching rules with interval-aware lexicographic comparisons are the de facto baseline.

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics: at every decision point, a priority function ranks the

operations that are ready to start and dispatches the best one, building a schedule in a single constructive pass. Classical rules prioritize by shortest or longest processing time (SPT/LPT), most operations or most work remaining (MOR/MWKR), earliest starting time, or combinations thereof; surveys catalogue more than a hundred variants [6, 7]. A step above single-attribute rules, the classical algorithm of Giffler and Thompson [5] restricts each choice to a machine's conflict set, guaranteeing *active* schedules and markedly improving plain rules when combined with a priority tie-break (the G&T-MWKR baseline of this paper). Decades of practice consolidated two lessons: rules are unbeatable in speed, transparency and ease of deployment, but hand-crafting them plateaued—no fixed combination of attributes performs well across shop configurations and objectives [7]. That plateau is precisely what motivated the *automated* design of rules.

2.3. Automated design of dispatching rules

GP hyper-heuristics evolve priority expressions over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [10, 11]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness, feature selection, and multi-objective formulations. Beyond single rules, a productive line combines several evolved rules into *ensembles* that jointly construct or vote on a solution [14], and recent work explores alternatives to evolutionary search for the same design task—systematic tree search over the space of expression trees [15]—reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. [12] evolve rules for dynamic job shops with uncertain processing times, exposing

summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimisation for the job shop [13], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes—and whose fitness is computed by interval arithmetic rather than by sampling realizations—has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [17, 18]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [13]. Both families occupy the same computational class at deployment (a single constructive pass); a controlled comparison on the IJSP is beyond the scope of this paper and is the object of ongoing work.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jk})$ exclusively and non-preemptively. The duration of operation o is an interval $p_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds.

Schedule construction uses interval arithmetic. For intervals $a = [\underline{a}, \overline{a}]$

and $b = [\underline{b}, \overline{b}]$, the two operations required by makespan minimisation are

$$a + b = [\underline{a} + \underline{b},\ \overline{a} + \overline{b}], \tag{1}$$

$$\max(a, b) = [\max(\underline{a}, \underline{b}),\ \max(\overline{a}, \overline{b})], \tag{2}$$

both component-wise [1]. A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of [1, 3], let $PM_o(\pi)$ and PJ_o denote the predecessors of task o in its machine (according to π) and in its job. The *semi-active* schedule [8] starts each task at

$$\mathbf{s_o}(\pi) = \max\left(\mathbf{c}_{PJ_o}(\pi),\ \mathbf{c}_{PM_o(\pi)}(\pi)\right), \qquad \mathbf{c_o}(\pi) = \mathbf{s_o}(\pi) + \mathbf{p_o}, \tag{3}$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_{R}\ \mathbf{C_{max}} \tag{4}$$

$$\text{s.t.}\quad \mathbf{C_{max}} = \max_{1 \le j \le n} \{\mathbf{c}_{o(j,m_j)}\}, \tag{5}$$

$$\underline{s}_{o(j,l)} \ge \underline{c}_{o(j,l-1)}, \quad \overline{s}_{o(j,l)} \ge \overline{c}_{o(j,l-1)}, \qquad 1 \le l \le m_j,\ 1 \le j \le n, \tag{6}$$

$$\underline{s}_o \ge \underline{c}_{o'} \vee \underline{s}_{o'} \ge \underline{c}_o, \quad \overline{s}_o \ge \overline{c}_{o'} \vee \overline{s}_{o'} \ge \overline{c}_o, \quad \forall o \ne o' : \mu(o) = \mu(o'), \tag{7}$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components [1, 3]. The minimum in (4) is taken with respect to an interval ranking R, since intervals admit no natural total order: we use the lexicographic order $\le_{Lex2}$ (upper bound first, lower bound as tie-break), one of the standard

rankings of the IJSP literature and the one found in [1] to yield the most robust schedules; the midpoint ranking $\leq_{MP}$—equivalent to the possibility-degree method of [9]—underlies the $E[\mathbf{C_{max}}]$ metric in which results are reported. All methods in this paper—the GP rules and every baseline—share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as relative error against the crisp Taillard lower bounds: $\mathrm{RE} = (E[C_{\max}] - LB)/LB \times 100$, where $E[C_{\max}]$ is the midpoint of the interval makespan.

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*—what an individual is and how it turns an IJSP instance into a schedule (Section 4.1)—and as an *evolutionary algorithm*—how the population of rules is initialized, varied and selected (Section 4.2).

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1; internal nodes from $\{+, -, \times, \div_p, \min, \max, \mathrm{neg}\}$ with protected division. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the N decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated—in a single vectorized pass—on the attribute vector of every $o \in \mathcal{E}$, and the operation with *lowest* priority is appended at its earliest feasible start on its machine, Eq. (3). Decoding a tree on an instance therefore costs exactly one constructive pass, the

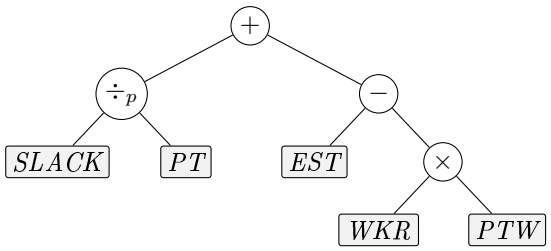

Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$). At each decision point the tree is evaluated on every eligible operation and the lowest-priority one is dispatched.

same as any hand-crafted rule. Classic rules are points of this search space—SPT is the tree PT and MWKR is neg(WKR)—and both are injected as seed individuals.

Protected division is defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

Algorithm 1 summarizes the generational loop, with the operators and parameters of Table 2. *Initialization* is ramped half-and-half over depths 2–4 (grow trees terminate early with probability 0.3 per node), with SPT and MWKR injected as two additional seed individuals. *Fitness* of a rule is the mean RE of its deterministic constructive pass over the four training instances—computed with the same environment, decoder and evaluator used by all baselines, which eliminates evaluator mismatch as a confound. *Selection* is by tournament; *variation* applies subtree crossover (a uniformly random node of parent A is replaced by a uniformly random subtree of par-

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine m: $p_o = [\underline{p}_o, \overline{p}_o]$ is its duration interval; C_j and C_m are the (interval) completion times of the last scheduled operation of job j and machine m; $\mathcal{R}_o$ is the set of operations of j after o; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\overline{p}_o$	worst-case processing time
PTW	$\overline{p}_o - \underline{p}_o$	duration uncertainty
EST	$\overline{e}_o = \max(\overline{C}_j, \overline{C}_m)$	worst-case earliest start
ESTW	$\overline{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_m)$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \overline{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\overline{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$\lvert\mathcal{R}_o\rvert$	operations remaining
SLACK	$\overline{e}_o - \min_{o' \in \mathcal{E}} \overline{e}_{o'}$	start slack within eligible set
ONE	1	constant

ent B) with probability p_c, and otherwise subtree mutation (a uniformly random node is replaced by a fresh grow tree of depth ≤ 3). *Replacement* is generational with elitism: the best individuals pass unchanged. Offspring exceeding the node cap receive fitness ∞ (bloat control).

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization: with probability ε=0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. Best-of-N then reports the lexicographically best schedule. This uniform perturbation of a single rule is the simplest member of a broader

Algorithm 1 GP hyper-heuristic for the IJSP

$P \leftarrow$ ramped half-and-half($pop - 2$) $\cup$ {SPT, MWKR}
evaluate(P) ▷ mean RE of one constructive pass on the training instances
for $g = 1$ **to** *gens* **do**
    $P' \leftarrow$ elite(P, e) ▷ elitism
    while $|P'| < pop$ **do**
        if rand() $< p_c$ **then**
            $child \leftarrow$ crossover(tournament(P), tournament(P))
        else
            $child \leftarrow$ mutate(tournament(P))
        end if
        $P' \leftarrow P' \cup \{child\}$ ▷ fitness ∞ if size $>$ cap
    end while
    $P \leftarrow$ evaluate(P')
end for
return best rule in P

family of randomized rule-based builders; a richer alternative, orthogonal to our contribution, swaps among a *set* of evolved rules during construction [16].

5. Experimental Setup

5.1. Benchmark instances

The main benchmark comprises the 70 interval versions of Taillard's instances (TA1–TA70, seven size classes from 15×15 to 50×20), with crisp Taillard lower bounds as the RE reference. Each interval instance is derived from its crisp counterpart with the generation scheme introduced in [1]: ev-

Table 2: Evolution parameters (conventional settings; see Section 7.5 for the sensitivity study).

Population / generations	100 / 50
Initialization	ramped half-and-half, depths 2–4 + {SPT, MWKR}
Selection	tournament, size 4
Crossover / mutation prob.	0.8 / 0.2 (subtree; mutation depth ≤ 3)
Elitism	2
Tree-size cap	40 nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

ery duration p is replaced by the integer interval $[p - \delta,\, p + \delta]$, with δ drawn uniformly in $[0,\, 0.15\,p]$ (maximum deviation 15%); machine routes are unchanged. The midpoint of every interval thus equals the original duration—the crisp instance is exactly the midpoint scenario of its interval version—which makes the crisp Taillard bounds a meaningful reference for $E[\mathbf{C_{max}}]$. After integer rounding, ≈20% of operations end up degenerate (deterministic), and the mean relative width is ≈13%. The interval Taillard set is the one used by the state-of-the-art tabu search [4], which makes that comparison an exact one.

As a secondary benchmark we use the 12 classical instances on which the published IJSP metaheuristics report results [1, 2, 3]: FT10, FT20, La21–La40 (selected), and ABZ7–9, in their interval versions generated with the same ±15% scheme and available from the authors' repository. These instances come from different families and sizes (10×10 to 20×15) than the Taillard set, and our rules are *not* re-evolved for them, so they double as a cross-family generalization test (Section 6.4).

5.2. Training protocol, data split and baselines

Rules are evolved on TA11–TA14 (20×15), and TA15–TA20 serve as development set: they never contribute to fitness, but design and configuration decisions (including the irace study of Section 5.3) were selected on their performance, so we flag them as subject to development reuse. The remaining 60 Taillard instances and the 12 classical instances are fully unseen by every design decision and constitute the honest generalization test.

Each evolution uses population 100 and 50 generations: ∼20,400 fitness evaluations (≈30 min single-core) per seed; thirty independent seeds per configuration, following standard practice in evolutionary computation.

Baselines are: fixed rules (SPT, LPT, MOR, MWKR) with interval-aware lexicographic comparison; the Giffler–Thompson active-schedule generator with rule tie-breaks (deterministic and ε-randomized inside the conflict set); and the published IJSP metaheuristics (GA [1], fEABC [2], ESABC [3], TS [4]) as yardsticks of the search-based computational class.

5.3. Hyperparameter configuration with irace

The four qualitative knobs of the evolution—tournament size, crossover probability, tree-size cap and elitism—were configured with irace over a budget of 200 experiments, each a full evolution (population 100, 50 generations) on the training instances whose cost is the resulting rule's RE on the development set, with the conventional configuration seeded. Table 3 lists the parameter space and the winning configuration. The elite configurations agree on a higher selection pressure than the default (tournament size 7 versus 4) and a crossover probability close to the default (0.72–0.77); the tree-size cap does not discriminate (surviving elites span caps of 20, 30 and 40). Throughout the race the statistical tests discard configurations only

Table 3: Hyperparameter space explored by irace (200 experiments; cost = development-set RE of the evolved rule) and resulting configurations.

Parameter	Range / values	Default (seeded)	irace winner
Tournament size	$\{2, 3, 4, 5, 7\}$	4	7
Crossover prob. p_c	[0.5, 0.95]	0.8	0.77
Tree-size cap	$\{20, 30, 40, 60\}$	40	30
Elitism	$\{1, 2, 4\}$	2	2

weakly, itself evidence that performance is largely flat over the hyperparameter space; the confirmation experiment of Section 7.5 quantifies this on unseen instances.

6. Results

6.1. Evolution behaviour and seed stability

Over 30 independent evolutions, training RE converges to 14.8 ± 0.9 (range 13.4–16.9), with most of the improvement realized by generation 25–30 (Figure 2). The evolved rules are structurally recognizable, combining the classical dispatching ingredients in interpretable ways. The best rule of the campaign (seed 27), for instance, is built from the ratio $SLACK/PT$ (slack per unit of processing time), work remaining terms bounded by operations-remaining counts, and an earliest-start correction—a refinement of the most-work-remaining/earliest-start family that hand-crafted rules approximate only coarsely.

6.2. Generalization to the full benchmark

Table 4 reports single-rollout RE per size class over all 70 instances, as mean $\pm$ standard deviation across the 30 evolved rules and for the single

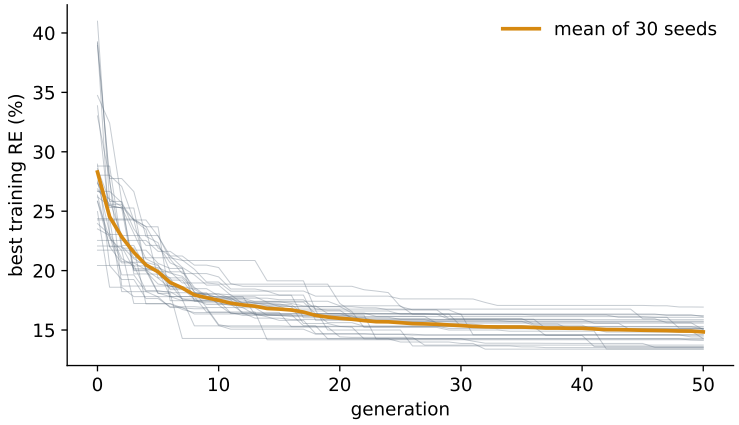

Figure 2: Best training fitness per generation for the 30 evolution seeds (default configuration): individual runs (grey) and their mean (amber).

Table 4: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean ± sd across the 30 independently evolved rules, and the best evolved rule.

	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
Mean of 30	17.6±1.2	17.5±1.2	20.8±1.4	21.2±1.2	25.2±1.2	13.5±1.4	15.0±1.2	18.7±0.8
Best rule	16.6	15.3	17.9	18.9	24.6	**13.3**	14.4	**17.3**

best rule. The best rule attains 17.3% global mean—evolved only on 20×15, its best class is the unseen 50×15 (13.3%), and the same pattern holds on average (13.5 ± 1.4). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

6.3. Position among constructive methods

All three constructive methods build one schedule per instance in a single pass: MOR averages 45.5% RE, G&T-MWKR 29.5%, and the evolved rules 18.7 ± 0.8 on average over the 30 evolutions (17.3% for the best rule). The gap is not marginal and not an artefact of averaging: the best evolved rule

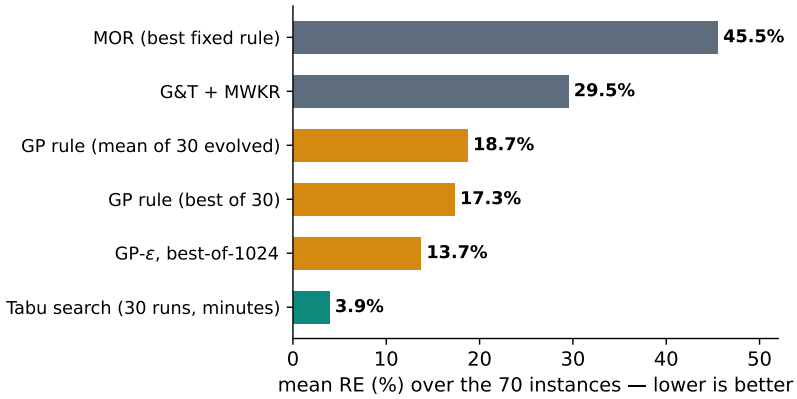

Figure 3: Constructive-method ladder on the 70 interval Taillard instances. Bars are single-solution methods except where noted.

is better than *both* baselines on every one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each baseline at $p < 10^{-6}$ ($z = -7.3$). Per-instance figures are given in Table A.8 (appendix). Crucially, this quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs the same, within measurement noise, as evaluating MOR or G&T-MWKR (the three differ by under 15% in per-schedule construction time in our implementation), since all reduce to one vectorized pass over eligible operations per decision. Both cost and quality thus place the evolved rule with the constructive methods, orders of magnitude below the search-based computational class in cost while closing much of the quality gap to it. Figure 3 places these results on the full method ladder, including the published tabu search [4] as the yardstick of the search-based class (an iterative search of minutes per instance versus a single constructive pass).

6.4. Cross-family generalization: the 12 classical instances

The 12 classical instances allow a direct confrontation with the *published* results of the IJSP metaheuristics, on their own benchmark and metric (Table 5). Recall that our rule was evolved on four interval Taillard 20×15 instances and is deployed here *zero-shot*, on different instance families and sizes. The single deterministic pass averages 13.9% RE—far ahead of MOR (45.8%) and G&T-MWKR (30.1%) in the same computational class. More strikingly, the randomized extension GP-ε at best-of-1024 reaches 9.9%, matching the published average of the genetic algorithm of [1] (9.8%)—a per-instance metaheuristic evolving a population for every instance—and beating it on half of the instances. The purpose-built ABC variants and the tabu search remain clearly ahead (5.5–6.4%), as expected of iterative search: the point of this comparison is not to compete with them but to locate the constructive frontier—a single evolved rule, transferred across families without retraining, now covers a substantial part of the gap that used to separate dispatching rules from metaheuristics.

6.5. Sampling behaviour (GP-ε)

Best-of-N over all 70 instances (pools of 1024 ε-randomized samples per instance): 18.6% (N=1, deterministic), 16.4% (16), 14.9% (64), 14.4% (256), 13.7% (1024)—about -4.9 points over ten doublings (Figure 4)—closing a substantial part of the gap to population-based methods while remaining a set of independent constructive passes, orders of magnitude below the iterative search in computational class and trivially parallel.

Table 5: RE (%) on the 12 classical interval instances. Own methods (deterministic single pass and GP-ε best-of-N; rule evolved on interval Taillard 20×15, applied zero-shot) versus published average results (30 runs) of the IJSP metaheuristics: GA [1] and the ABC variants [2, 3]. LB is the best-known lower bound of the expected makespan.

		Own (zero-shot)			Published (avg of 30)			
Inst.	LB	GP	GP-ε_{64}	GP-ε_{1024}	GA	ABC$_{E3}$	fEABC	ESABC
FT10	930	8.1	7.0	6.3	5.2	4.1	3.5	3.0
FT20	1165	9.1	7.6	6.2	4.4	1.7	1.8	1.8
La21	1046	14.1	12.5	9.6	5.0	5.0	4.2	4.0
La24	935	11.1	8.7	7.4	6.3	5.1	4.9	5.0
La25	977	11.3	11.3	6.6	5.1	3.9	3.4	2.7
La27	1235	16.0	9.7	9.2	10.2	4.7	4.6	4.1
La29	1152	14.3	12.2	11.4	14.2	8.6	7.4	7.0
La38	1196	14.3	10.4	8.0	9.2	6.9	6.1	5.8
La40	1222	13.0	12.3	8.1	8.7	4.2	4.6	4.1
ABZ7	656	16.2	13.3	11.5	12.5	7.3	6.6	6.7
ABZ8	645	19.9	18.8	16.1	18.5	12.1	11.1	10.9
ABZ9	661	19.6	18.9	18.8	18.0	13.1	11.6	11.2
Mean		13.9	11.9	9.9	9.8	6.4	5.8	5.5

7. Analysis

Beyond aggregate quality, this section examines *why* the evolved rules work: what they are made of, whether the interval-specific ingredients actually contribute, how robust the resulting schedules are under executional uncertainty, whether they survive re-evaluation under other uncertainty models, and how sensitive the method is to its own hyperparameters.

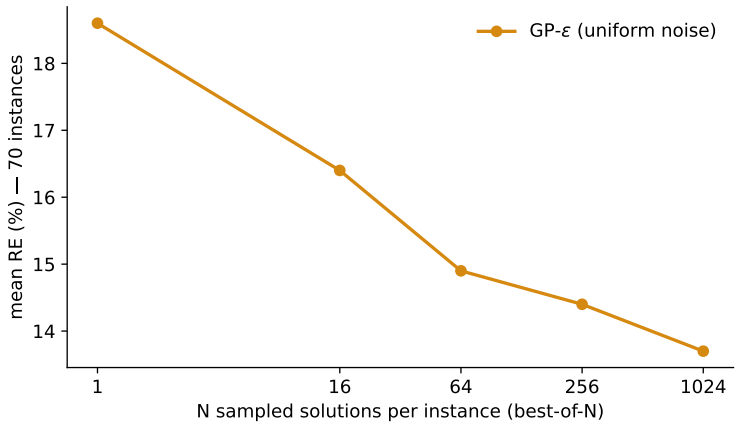

Figure 4: Best-of-N curve of the evolved rule with uniform ε-randomization (ε=0.1): mean RE over the 70 instances as the sampling budget doubles.

7.1. Rule anatomy and interpretability

A practical appeal of GP over neural policies is that its output is a readable expression. Evolved rules are compact—mean tree size 33 nodes (19–40), mean depth 10—so a rule can be inspected and simplified by hand. Figure 5 reports how often each terminal is used across the 30 evolved rules. The backbone is the classical scheduling triad—slack ($SLACK$), work remaining (WKR) and earliest start (EST)—together with worst-case processing time (PT), confirming that the evolution rediscovers and refines known dispatching logic rather than exploiting benchmark artefacts. The *interval-width* terminals (PTW, $ESTW$, $WKRW$) account for ≈21% of all terminal usage and appear in 28 of the 30 evolved rules: the evolution consistently reaches for uncertainty information, even though worst-case bounds carry the larger share of the signal. Whether that width information translates into better schedules is quantified by the ablation of Section 7.2 and the robustness analysis of Section 7.3.

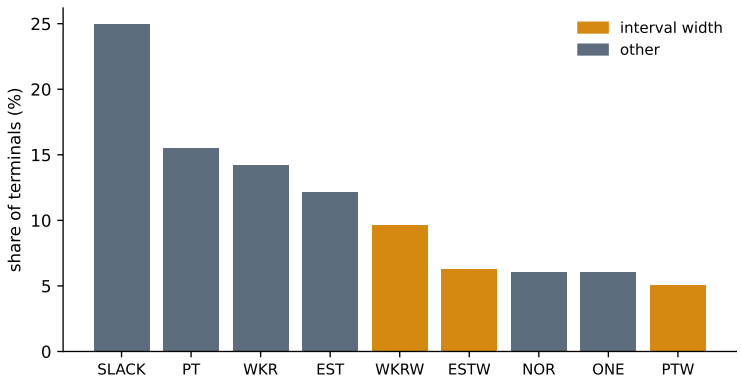

Figure 5: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

7.2. The value of interval-awareness: terminal ablation

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution. We re-run the full evolution—30 independent seeds per configuration—with and without the width terminals (PTW, $ESTW$, $WKRW$) in the terminal set, everything else held fixed, and compare the resulting mean RE over the 70 instances with a paired Wilcoxon signed-rank test on the shared seeds. [TODO: rellenar con la campaña 30×3: media±std no-width vs full, ΔRE, y el estadístico de Wilcoxon + el experimento de dos regímenes (fitness robusto) cuando termine la campaña.]

7.3. Robustness under executional uncertainty

Relative error measures a-priori quality, but for the IJSP a schedule is ultimately *executed* under one realization of the durations. A good predictive schedule is one whose executed makespan stays close to its a-priori prediction. We quantify this with the $\bar{\varepsilon}$ robustness of [1]: for a schedule with fixed processing order and predicted expected makespan $E[C_{\max}]$

(interval midpoint), we draw K=1000 realizations of the durations (uniform on each operation's interval), execute the fixed order, and average the relative deviation of the executed makespan $C^{ex}_{\max}$ from the prediction, $\bar{\varepsilon} = \frac{1}{K}\sum_k |C^{ex,k}_{\max} - E[C_{\max}]|/E[C_{\max}]$. Lower is more robust. To probe behaviour under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

The evolved rule does not merely produce lower-makespan schedules—it produces *more robust* ones, on two complementary measures (Table 6). First, its predicted makespan interval is the *tightest*: a relative width of 12.6% against 13.8% (G&T-MWKR) and 14.4% (MOR), so the a-priori uncertainty attached to the schedule is smaller. Second, and more tellingly, its executed makespan tracks the prediction most closely: $\bar{\varepsilon}$ is lower than for both baselines at the nominal width (5.8 versus 6.5 for MOR and 6.3 for G&T-MWKR), and its advantage *widens* as uncertainty grows (at +40%: 7.7 versus 8.9 and 8.6). Because the rule can read interval widths, it favours dispatching decisions whose realized outcome is less sensitive to how the durations materialize. Figure 6 shows the per-instance $\bar{\varepsilon}$ distributions.

[TODO: tras la campaña: rehacer con la mejor regla de 30 semillas y anadir el brazo no-width (robustez interval-aware vs ablacion); test estadistico.]

7.4. Transfer to other uncertainty models

The interval representation is one of several coexisting formalisms for duration uncertainty; deterministic (crisp) and fuzzy models are equally established [19]. A natural concern is whether schedules built by a rule evolved *for intervals* remain good when the durations are ultimately handled under a

Table 6: Robustness over the 70 instances (lower is better on both measures). *Rel. width*: relative width of the predicted makespan interval, $(\overline{C}_{\max} - \underline{C}_{\max})/E[C_{\max}]$ (%). $\bar{\varepsilon}$ ($\times 10^3$): mean deviation of the executed makespan from its prediction, at nominal interval widths and widened by +20% and +40% (Monte Carlo, K=1000 realizations, common random numbers across methods).

		$\bar{\varepsilon}$ ($\times 10^3$)		
Method	Rel. width (%)	+0%	+20%	+40%
GP rule	**12.6**	**5.8**	**6.8**	**7.7**
G&T-MWKR	13.8	6.3	7.5	8.6
MOR	14.4	6.5	7.7	8.9

different model. We test this by re-decoding the same processing orders under two alternative arithmetics, without any re-evolution: (i) *crisp*, executing each sequence with the original Taillard durations (the midpoint scenario of each interval instance); and (ii) *triangular fuzzy* (TFN), where each duration becomes the triangle $(\underline{p}, p, \overline{p})$ and schedules are built with fuzzy arithmetic and ranked by the expected value $E[\tilde{C}] = (a+2b+c)/4$. The evaluation uses the GP-ε pools of Section 6 (1024 sequences per instance, all 70 instances), so both quality and *ranking* transfer can be measured.

Transfer is essentially lossless (Table 7). The best sequence per pool scores 13.0% RE under crisp and 13.2% under TFN evaluation, in line with its interval quality; pool means are unchanged (21.0–21.2 versus 21.4); and the interval ranking of each pool is preserved almost exactly under both models (mean Spearman correlation 0.994 crisp, 0.998 fuzzy)—the best solution under interval arithmetic is, for practical purposes, also the best crisp and fuzzy solution. Notably, there is no "robustness tax": evaluating crisp, the interval-selected sequences are marginally *better* than their interval RE

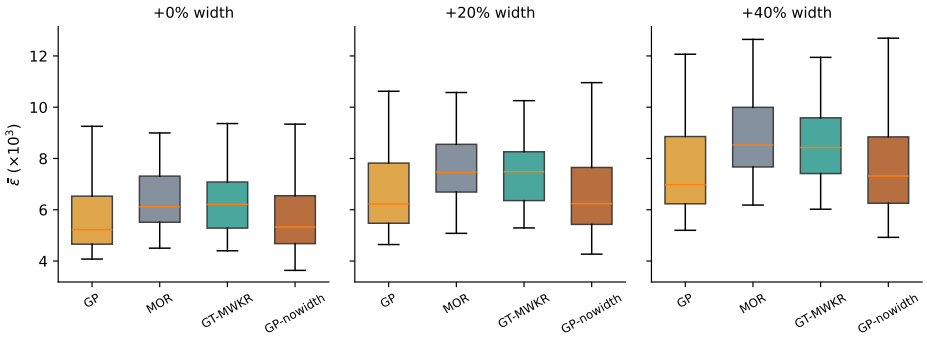

Figure 6: Per-instance $\bar{\varepsilon}$ robustness by method at each uncertainty level (lower is better). The evolved rule's robustness advantage over the constructive baselines grows with interval width.

Table 7: Transfer of the GP-ε pools (1024 sequences per instance, 70 instances) to other uncertainty models: the same processing orders re-decoded under each arithmetic. Spearman: mean per-instance rank correlation between the interval ranking and the re-decoded ranking.

Evaluation model	Best of pool (%)	Pool mean (%)	Spearman
Interval (native)	13.4	21.4	—
Crisp (midpoint scenario)	13.0	21.0	0.994
Triangular fuzzy, $E[\tilde{C}]$	13.2	21.2	0.998

suggests. For a practitioner this means the choice of uncertainty formalism can be deferred: schedules produced by the interval-evolved rule survive re-evaluation under the other standard models of the literature.

7.5. Hyperparameter confirmation

Under the same pre-registered confirmation applied to every design decision—the winning configuration re-evolved over 30 independent seeds and each resulting rule evaluated on all 70 instances—the tuned configuration brings

no significant improvement: 18.99 ± 1.33 mean RE versus 18.68 ± 0.84 for the untuned default, with a paired Wilcoxon signed-rank test on the shared seeds far from significance ($z = -0.96$); if anything, the stronger selection pressure increases seed-to-seed variance. The one systematic effect of the tuned configuration is *structural*: with the same performance, it evolves markedly smaller trees (26.4 versus 33.1 nodes on average), i.e., the tuning budget buys readability rather than quality. We therefore characterize the method as robust to its evolutionary hyperparameters: conventional settings are statistically indistinguishable from the irace winner on unseen instances.

7.6. Limitations

Three limitations bound the scope of our conclusions. First, all rules operate inside a semi-active SGS; active (insertion-based) generation schemes are known to help weaker constructors, and although our evolved rules produce near-active schedules already, co-evolving rules with an active SGS in the loop remains untested. Second, the uncertainty model is the $\pm 15\%$ symmetric interval scheme standard in the IJSP literature; other widths and asymmetric intervals may change the relative value of uncertainty information. Third, our runtime figures are relative (evolved rules cost the same as hand-crafted ones per pass); absolute times depend on the research-grade Python implementation and would shrink by orders of magnitude in an optimized production implementation, without affecting any ranking reported here.

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations, built on a terminal

set that exposes interval widths as first-class attributes. The empirical conclusions are three. (i) Evolved rules dominate every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.7 ± 0.8 over 30 evolutions, 17.3% for the best rule, versus 29.5% for the strongest baseline—which the best rule beats on all 70 instances) at the cost of a single constructive pass, and (ii) they generalize zero-shot across all size classes—evolved on 20×15, best on the unseen 50×15—because the representation contains no size-bound quantities. (iii) The method is robust to its evolutionary hyperparameters: a 200-experiment irace study confirmed over 30 evolutions per configuration yields no significant gain over conventional settings, only systematically smaller trees at equal performance.

Future work follows three lines. First, richer symbolic models: ensembles of complementary rules [14] and co-evolution of construction and tie-breaking. Second, using evolved rules as cheap, high-quality population initializers for the metaheuristics that define the state of the art in the IJSP, where preliminary pools built from randomized rule rollouts are already compatible with the seeding protocol of [4]. Third, a controlled comparison with learned neural constructive policies under matched training and inference budgets—the head-to-head evaluation that the recent literature [13] identifies as missing.

Reproducibility

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. [TODO: URL/DOI del

repositorio al publicar]

Appendix A. Per-instance results

Table A.8 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the three deterministic constructive methods.

[TODO: tabla por instancia: se refresca con la mejor regla de la campana 30x3]

References

[1] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela. A genetic approach to the job shop scheduling problem with interval uncertainty. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*, pages 663–676. Springer, 2020.

[2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.

[3] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.

[4] H. Díaz et al. Neighbourhood structures and ranking operators for the interval job shop scheduling problem. Under review, 2026.

[5] B. Giffler, G. L. Thompson. Algorithms for solving production-scheduling problems. *Operations Research*, 8(4):487–503, 1960.

[6] S. S. Panwalkar, W. Iskander. A survey of scheduling rules. *Operations Research*, 25(1):45–61, 1977.

[7] R. Haupt. A survey of priority rule-based scheduling. *OR Spektrum*, 11(1):3–16, 1989.

[8] A. Sprecher, R. Kolisch, A. Drexl. Semi-active, active, and non-delay schedules for the resource-constrained project scheduling problem. *European Journal of Operational Research*, 80(1):94–102, 1995.

[9] D. Lei. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 62(4):1200–1208, 2012.

[10] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, 20(1):110–124, 2016.

[11] S. Nguyen, Y. Mei, M. Zhang. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, 3(1):41–66, 2017.

[12] D. Karunakaran, Y. Mei, G. Chen, M. Zhang. Dynamic job shop scheduling under uncertainty using genetic programming. In *Intelligent and Evolutionary Systems*, pages 195–210. Springer, 2017.

[13] M. Xu, Y. Mei, F. Zhang, M. Zhang. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, 58:160, 2025.

[14] F. J. Gil-Gala, C. Mencía, M. R. Sierra, R. Varela. Learning ensembles

of priority rules for online scheduling by hybrid evolutionary algorithms. *Integrated Computer-Aided Engineering*, 28(1):65–80, 2021.

[15] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, R. Varela. Tree search hyper-heuristic with application to combinatorial optimization. *Journal of Heuristics*, 31:25, 2025.

[16] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, J. Puente. Greedy randomised adaptive solution builder using priority rules. In *Proc. Genetic and Evolutionary Computation Conference Companion (GECCO '25)*, pages 211–214. ACM, 2025.

[17] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.

[18] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.

[19] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278:81–97, 2015.

Table A.8: Per-instance RE (%) of the constructive methods (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	MOR	G&T	GP	Inst.	LB	MOR	G&T	GP
TA1	1231	29.7	26.1	18.6	TA36	1819	57.4	42.8	20.0
TA2	1244	34.8	27.8	15.5	TA37	1771	43.0	35.6	23.4
TA3	1218	42.9	33.2	16.1	TA38	1673	50.6	28.0	21.1
TA4	1175	62.6	27.7	17.0	TA39	1795	50.2	28.9	18.7
TA5	1224	50.7	33.0	8.8	TA40	1651	42.7	42.3	21.5
TA6	1238	36.6	18.2	16.7	TA41	1906	53.5	45.0	32.3
TA7	1227	39.4	26.1	21.8	TA42	1884	61.3	38.1	23.0
TA8	1217	41.3	25.0	13.9	TA43	1809	61.4	39.8	24.2
TA9	1274	38.2	28.2	17.9	TA44	1948	58.8	34.7	24.4
TA10	1241	38.0	22.1	19.6	TA45	1997	59.5	33.7	16.2
TA11	1357	49.9	30.7	16.5	TA46	1957	59.7	31.6	24.5
TA12	1367	47.8	34.6	15.4	TA47	1807	78.3	30.0	23.2
TA13	1342	46.5	26.9	14.9	TA48	1912	52.2	33.7	26.3
TA14	1345	44.1	29.7	9.7	TA49	1931	46.9	35.3	22.8
TA15	1339	51.3	30.6	16.4	TA50	1833	56.2	46.2	29.7
TA16	1360	35.8	34.0	18.5	TA51	2760	38.2	34.7	26.3
TA17	1462	50.1	17.7	12.8	TA52	2756	38.1	26.5	10.5
TA18	1377	54.2	31.8	18.0	TA53	2717	28.2	18.3	11.0
TA19	1332	43.2	29.3	13.7	TA54	2839	23.6	16.1	4.7
TA20	1348	43.7	24.1	16.9	TA55	2679	37.6	29.2	13.9
TA21	1642	40.8	28.0	15.4	TA56	2781	44.8	19.8	10.9
TA22	1561	46.3	30.8	24.6	TA57	2943	29.9	20.0	12.7
TA23	1518	45.2	35.3	22.5	TA58	2885	30.5	26.3	13.7
TA24	1644	36.2	23.8	14.4	TA59	2655	45.2	28.2	19.0
TA25	1558	53.2	33.8	17.4	TA60	2723	25.3	19.8	10.1
TA26	1591	52.4	31.2	18.0	TA61	2868	48.7	25.4	17.2
TA27	1652	49.8	31.1	16.9	TA62	2869	43.5	27.6	14.9
TA28	1603	38.8	26.8	17.7	TA63	2755	46.0	23.8	11.8
TA29	1583	56.4	32.0	12.3	TA64	2702	43.3	28.4	11.5
TA30	1528	53.3	36.1	19.8	TA65	2725	44.7	30.0	19.7
TA31	1764	34.4	34.4	15.2	TA66	2845	38.1	28.0	14.8
TA32	1774	55.7	33.3	21.2	TA67	2825	58.4	26.1	12.7
TA33	1788	41.6	34.8	22.7	TA68	2784	45.8	19.1	10.3
TA34	1828	34.8	31.8	16.4	TA69	3071	38.0	22.8	11.4
TA35	2007	30.5	24.1	9.2	TA70	2995	52.3	23.4	20.2